

Parallel and Distributed Systems (SPM) Questions and Answers

Academic Year 2025/2026 — Filtered Syllabus (V2)

This document contains the official collection of exam questions and answers for the course *Parallel and Distributed Systems: Paradigms and Models*, filtered to match the current academic year 2025/2026 syllabus (excluding non-covered topics such as the FastFlow framework or Control Parallelism).

1. Performance Metrics and Fundamental Laws

Q: What is Latency?

A: It is the time spent for executing a single task in a computation.

Q: What is Completion Time?

A: The time spent since the beginning of the execution of the first task through the end of the last task.

Q: What is Throughput?

A: The amount of tasks computed per unit of time.

Q: What is Service Time?

A: Service time refers to the duration required to produce the results of a task within a sequence (stream), starting immediately after the results of the preceding task have been delivered.

Q: What does Amdahl's Law state?

A: Amdahl's Law states that the sequential (non-parallelizable) fraction of a program limits the maximum speedup that can be achieved by parallelizing it, regardless of the number of processors used.

Q: What is Gustafson's Law?

A: Gustafson's Law focuses on weak scaling, where the problem size increases with the number of processors. It assumes that the time spent in the parallel part scales with the number of processors, providing a more realistic evaluation of scaled speedup for high-performance computing systems compared to Amdahl's law.

2. Forms of Parallelism and Structured Patterns

Q: What is Data Parallelism?

A: Data Parallelism is achieved by dividing an initial task into independent subtasks operating on different portions of the input data; their partial results are then combined to produce the final outcome. The effort required to partition the task and merge the results constitutes the computational overhead. In embarrassingly parallel computations, the task is easily divided into completely independent subtasks with straightforward merging. The main challenge lies in balancing the time spent splitting tasks and managing parallel execution against the performance gains.

Q: What is Stream Parallelism?

A: Stream Parallelism involves the parallel execution of tasks from an input stream, where tasks become available at different times (they are not all known at the beginning). The interarrival time between tasks significantly impacts the parallel computing potential. If there are no dependencies between the tasks of the stream, the computation is embarrassingly parallel. Stream parallelism does not reduce the execution time of individual tasks (latency), but it increases the overall throughput of the application, thereby reducing the total completion time. The stream of tasks can be theoretically infinite.

Q: What is the difference between Data Parallelism and Stream Parallelism?

A: Data parallelism aims to reduce the execution time (latency) of a single task by dividing its data into smaller chunks processed concurrently. Conversely, stream parallelism does not reduce the latency of an individual task, but instead improves the overall throughput of the application by processing multiple independent stream items concurrently as they arrive over time.

Q: What is Task Parallelism?

A: Task Parallelism consists of decomposing a computation into multiple distinct tasks that may have mutual dependencies. Tasks without dependencies can be executed in parallel. Two tasks are dependent if one cannot start until the other finishes, usually because the output of the first is needed as input for the second. Task parallelism reduces the total latency by enabling concurrent execution of independent tasks.

Q: How is Task Parallelism represented?

A: It is modeled using a Directed Acyclic Graph (DAG), where each node represents a task to be executed and each directed edge expresses a logical or data dependency between two tasks. The initial node has no incoming dependencies.

Q: How can Data Parallel and Stream Parallel computations be modeled using Task Parallelism?

A:

- **Data Parallel:** Can be modeled with an initial task that splits the data into partitions, a set of parallel tasks that process each partition, and a final collection (merge) task that reassembles the outputs. There are dependency edges from the split to the processing tasks, and from those to the merge task.

- **Stream Parallel:** Can be modeled by creating and instantiating a continuous sequence of independent tasks, one for each data element or block arriving from the input stream.

Q: Does the concept of Structured and Unstructured Programming coincide between sequential and parallel programs?

A: No, it differs significantly. In sequential programs, structured programming organizes the control flow using well-defined constructs (if-else, loops, functions), eliminating chaotic GOTO statements. In parallel programs, **Structured Parallel Programming** refers to models where parallelism is implemented using predefined patterns or algorithmic skeletons (e.g., OpenMP constructs like parallel for, or high-level patterns like pipeline and farm), delegating concurrency management to runtimes and compilers. Unstructured parallel programming relies on low-level abstractions (naked threads like std::thread, mutexes, semaphores), requiring the programmer to manually manage synchronization, mapping, and coordination.

Q: What are the advantages of structured parallel programs compared to unstructured ones?

A: They offer greater ease of design and implementation because critical responsibilities (such as synchronization and load balancing) are shifted from the programmer to the runtime environment. This enables automatic optimization (auto-tuning) and ensures much higher code and performance portability across different hardware architectures.

Q: What are "shapes" (patterns) in parallel programming?

A: A shape (or pattern) is a structured composition of parallel computations with well-defined dependencies that require standard orchestration to produce the final result. They are mainly divided into stream patterns and data patterns.

Q: What are the main Stream Parallel Shapes?

A: They operate on input streams to produce output streams. The main ones are:

- **Pipeline:** Computes a composition of consecutive functions/stages ($f_1, f_2, \dots, f_k$) in parallel across all elements of the stream. Parallelism is exploited by processing different stages on different stream elements at the same time.
- **Farm:** Applies the same function f to all elements of an input stream by replicating Workers. It is embarrassingly parallel since each stream element can be processed completely independently. The output order may not be preserved unless requested.

Q: What are the main Data Parallel Shapes?

A: They operate on a defined collection of data to produce a new collection or an aggregated result:

- **Map:** Applies a function f to each element of a collection C independently, producing an isomorphic collection of the same shape and size. It is embarrassingly parallel.
- **Reduce:** Aggregates the elements of a collection using an associative and commutative binary operator, reducing them to a single scalar value. It can be parallelized using a

tree-structured computation.

- **Prefix (Scan):** Computes the vector of partial sums of a collection using an associative binary operator.
- **Stencil:** Updates each element of a collection by combining it with its neighboring elements (neighborhood) according to a predefined function. The processing is parallel, but requires access to the original immutable values during computation.
- **Divide & Conquer:** Resolves a problem by recursively dividing it into smaller subproblems, solving them (when a base case is reached), and combining the partial results. Parallelism is expressed by executing recursive branches concurrently.

Q: What is the difference between Map and Farm?

A: Map is a data-parallel pattern working on a static collection of data, producing an isomorphic structure and maintaining element correspondence. Farm is a stream-parallel pattern operating on a dynamic stream of tasks over time, where workers process stream items as they arrive, and the output order or structure may differ from the input.

Q: How do you choose the optimal number of Workers in a Farm using Service Time?

A: Interpreting the Farm by its logical components (Emitter, Workers, Collector), the overall Service Time is governed by the bottleneck: $\text{Service_Time}(\text{Farm}) = \max(t_E, t_W / n_w, t_C)$, where t_E is the Emitter time, t_C is the Collector time, and t_W is the computation time of a single task by a Worker. To optimize performance and prevent Workers from becoming the bottleneck, their temporal component should not be the maximum, meaning: $t_W / n_w \leq \max(t_E, t_C)$. Consequently, the minimum number of workers required is: $n_w > \lceil t_W / \max(t_E, t_C) \rceil$.

Q: What is meant by finding the Normal Form?

A: In the context of refactoring and optimizing structured programs based on algorithmic skeletons, it consists of rewriting and reorganizing the pattern composition tree (e.g., eliminating redundant nestings of pipelines or farms) to bring it into an optimal standardized structure that maximizes runtime efficiency.

3. Hardware Optimization and Shared Memory

Q: What characteristics of modern processors are exploited for parallelism?

A: Modern processors exploit multiple levels of parallelism:

- **Multiple Cores:** MIMD parallelism via integrating multiple cores that execute threads independently and asynchronously.
- **Vector Units (SIMD):** Exploit data-level parallelism by executing the same vector instruction on multiple elements simultaneously within each core.

- **Instruction Level Parallelism (ILP):** Exploit parallelism at the instruction level using hardware techniques such as pipelining and superscalar execution.

Q: What is the correct way to concurrently implement Matrix-Vector Multiplication to reduce cache conflicts?

A: The optimized algorithm distributes the rows of the matrix among threads. To avoid overhead due to continuous writes into shared memory, each thread accumulates the partial product inside a temporary local variable (stored in a register or the private stack) and updates the global shared vector cell only once at the end of the inner row loop:

```
for row = id; row < max_rows; row += num_threads do
  res = 0;
  for col = 0; col < n; col++ do
    res += mtx[row][col] * vec[col];
  end for
  b[row] = res; // Single final write
end for
```

Q: What is False Sharing, and how do coherence protocols or programmers fix it?

A: False Sharing occurs when multiple threads executing on different cores modify separate, independent variables that happen to reside within the same cache line. This causes continuous and unnecessary cache line invalidations between cores, degrading performance. Solutions include:

- **At Coherence Protocol / Hardware Level:** Alignment mechanisms and cache state transitions (MESI/MOESI).
- **At Programming (User) Level:** Inserting hardware-specific padding (e.g., 64 bytes) between data to force them onto distinct cache lines, or minimizing shared memory accesses by accumulating intermediate results in private local variables/registers for a single final write.

4. Parallel Computing Models

Q: How do Work and Span metrics function in the Work-Span model?

A: Representing a parallel computation as a Directed Acyclic Graph (DAG):

- **Work (T_1):** The total time required to execute all operations in the graph on a single sequential processor.
- **Span (T_∞):** The minimum time required to complete the longest chain of sequential dependencies (the critical path) within the graph, assuming an infinite number of

processors. The maximum ideal speedup is upper-bounded by the ratio T_1 / T_∞ .

Q: What is the difference between the Work-Span model and Amdahl's Law regarding Speedup?

A: The Work-Span model evaluates the intrinsic limits of parallelism based on the topological structure of dependencies (critical path) of the application graph. Amdahl's Law, on the other hand, defines the maximum speedup limit based rigidly on the fixed fraction of purely sequential code relative to the total work duration, without explicitly modeling dependency graphs.

Q: What is the "Combining Concurrent Write" mode in the PRAM model?

A: In the theoretical PRAM (Parallel Random Access Machine) model, the Combining concurrent write policy specifies that when multiple processors attempt to write simultaneously to the exact same memory cell, the written values are combined into a single value using an associative and commutative binary operation (such as sum, product, minimum, or logical AND).

Q: How is the BSP (Bulk Synchronous Parallel) model structured and how does it analyze costs?

A: The BSP model describes a distributed-memory system and is characterized by four main parameters: $BSP(p, r, g, l)$, where p is the number of processors, r is the computing speed (FLOPS), g is the communication cost per data word, and l is the cost of a global synchronization barrier. A BSP algorithm is organized into a sequence of **Supersteps**, separated by global barriers, divided into:

- Computational Superstep: where each processor performs local computations using only its local data.
- Communication Superstep: where nodes exchange messages over the network. Message exchange is modeled as an *h-relation*, where each processor sends or receives at most h words of data. The communication cost of the superstep is proportional to $h * g$.

Q: What is meant by Data-Flow and Macro Data-Flow?

A: It is an approach to parallelism driven exclusively by data dependencies. The computation is a graph where nodes are operations and edges are read-after-write (RAW) dependencies. In standard Data-Flow, nodes represent individual elementary instructions. In **Macro Data-Flow**, nodes represent entire complex sequential functions or structured blocks of code that activate as soon as all their input data becomes available.

5. Resource Management with SLURM

Q: What are Partitions in SLURM and how are they inspected?

A: SLURM organizes cluster compute nodes into logical partitions (equivalent to submission queues). Partitions can be configured with specific constraints, such as maximum job run times

or limits on the maximum number of allocatable nodes. To view available partitions on the cluster and their current status, the `sinfo` command is used. Jobs are then distributed across nodes based on priority criteria and backfilling policies.

6. Distributed Programming with MPI

Q: In which systems is the MPI standard used?

A: It is used in distributed-memory systems, such as clusters or supercomputers composed of multiple independent compute nodes interconnected by a high-performance communication network, where each node has its own private address space. The de facto standard for managing communication in these systems is MPI (Message Passing Interface), with OpenMPI being a widely used implementation.

Q: How is time measured in an MPI application?

A: MPI provides a high-resolution timing function called `MPI_Wtime()`, which returns the elapsed time in seconds relative to an arbitrary point in the past. It is invoked at the start and end of the code section to be monitored to calculate the difference.

Q: How is the clean termination of a Master-Worker application properly managed in MPI?

A: The Master process must receive a special end-of-stream message, commonly called an EOS (End-of-Stream) message, from all Worker processes. To identify this message, the Master checks if the packet is associated with a specific termination tag (e.g., a numeric eos tag defined via an enum). When all processes have completed their activities and synchronized the flow, the global clean-up function `MPI_Finalize()` is called to safely close the MPI environment before the program exits.

Q: Which MPI functions are used to create custom or non-contiguous data structures in memory?

A: MPI allows defining complex derived datatypes to optimize transmissions and avoid manual serialization:

- `MPI_Type_vector`: Allows referencing blocks of data of fixed size equally spaced by a constant stride, useful for sending columns or sections of row-major matrices.
- `MPI_Type_struct`: The most general and flexible constructor; it allows combining heterogeneous data types arranged at arbitrary offsets in memory to fully describe native C/C++ structures.